

-> Click Here For Video Lectures ; YT @AnishSaha_

Gate-Level Modeling

Array of Instances

Array of gates

When multiple identical gates are required, an **array of instances** simplifies their declaration. Instead of writing repetitive instantiations, Verilog allows defining arrays of primitive instances.

This approach is efficient and reduces coding effort, especially for large designs.

```
module array_of_instances;
    wire [7:0] out1,out2, in1, in2;
    assign in1 = 'b'd1;
    assign in2 = 'b'd2;
    initial #1 $display("in1 : %b, in2 : %b",in1,in2);
    // Array of 8000 gates
    nand n_gate[7:0](out1, in1, in2);
    // Equivalent to:
    nand n_gate0(out1[0], in1[0], in2[0]);
    nand n_gate1(out1[1], in1[1], in2[1]);
    nand n_gate2(out1[2], in1[2], in2[2]);
    nand n_gate3(out1[3], in1[3], in2[3]);
    nand n_gate4(out1[4], in1[4], in2[4]);
    nand n_gate5(out1[5], in1[5], in2[5]);
    nand n_gate6(out1[6], in1[6], in2[6]);
    nand n_gate7(out1[7], in1[7], in2[7]);
    initial #5 $display("out1 : %b, out2 : %b",out1,out2);
    initial #10 $finish;
endmodule
```

YouTube - PrepFusion - GATE

Gate-Level Modeling

Gate-Level Multiplexer Design

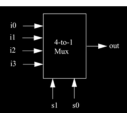
In this section, we discuss the design of a **4-to-1 multiplexer** using logic gates. A multiplexer connects multiple data inputs to a single output based on select signals and can also be used to implement Boolean functions. For this example, we use two select signals (s1 and s0) and assume they do not take on x (unknown) or z (high impedance) values.

I/O Diagram and Truth Table

The I/O diagram of the 4-to-1 multiplexer, which helps set up the port list, is depicted below. Each combination of the select signals determines which input (i0, i1, i2, i3) is connected to the output (out).

Logic Diagram

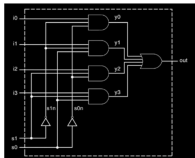
The logic diagram for the multiplexer uses basic gates and directly maps to its Verilog implementation. Intermediate signals like s1n and s0n (complements of s1 and s0) and internal nets (y0, y1, y2, y3) are employed to construct the circuit.



s1	s0	out
0	0	i0
0	1	i1
1	0	i2
1	1	i3

Gate-Level Modeling

```
// Module for 4-to-1 Multiplexer
module mux_4to1 (out, i0, i1, i2, i3, s1, s0);
    // Port declarations
    output out;
    input i0, i1, i2, i3;
    input s1, s0;
    // Internal wire declarations
    wire y0, y1, y2, y3;
    // Generate components of select signals
    not n0(s0, s0n);
    not n1(s1, s1n);
    // 8-input and gates for each input
    and y0(s0n, s1n, i0);
    and y1(s0n, s1, i1);
    and y2(s1n, s0, i2);
    and y3(s1, s0, i3);
    // Combine outputs using a 4-input OR gate
    or out(y0, y1, y2, y3);
endmodule
```



```
// Testbench for 4-to-1 Multiplexer
module stimulus;
    // Declare inputs and outputs
    reg i0, i1, i2, i3;
    reg s1, s0;
    wire out;
    // Instantiate the multiplexer
    mux_4to1 my_mux(out, i0, i1, i2, i3, s1, s0);
    // Apply test cases
    initial begin
        i0 = 1; i1 = 0; i2 = 1; i3 = 0;
        // Test each select signal combination
        $display("i0=%b, i1=%b, i2=%b, i3=%b", i0, i1, i2, i3);
        s1 = 0; s0 = 0; #1 $display("s1=%b, s0=%b, out=%b", s1, s0, out);
        s1 = 0; s0 = 1; #1 $display("s1=%b, s0=%b, out=%b", s1, s0, out);
        s1 = 1; s0 = 0; #1 $display("s1=%b, s0=%b, out=%b", s1, s0, out);
        s1 = 1; s0 = 1; #1 $display("s1=%b, s0=%b, out=%b", s1, s0, out);
    end
endmodule
```

```
i0=1, i1=0, i2=1, i3=0
s1=0, s0=0, out=0
s1=0, s0=1, out=0
s1=1, s0=0, out=1
s1=1, s0=1, out=0
```

YouTube - PrepFusion - GATE

Gate-Level Modeling

4-Bit Ripple Carry Full Adder

A **4-bit ripple carry full adder** is designed using four 1-bit full adders. It computes the sum and carry-out for 4-bit inputs with a carry-in signal.

1. 1-Bit Full Adder:

Equations:

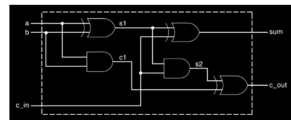
- sum = a ⊕ b ⊕ cin
- cout = (a + b) + (cin · (a ⊕ b))

Logic Diagram:

- Includes XOR, AND, and OR gates.

Verilog

```
module fulladd (sum, c_out, a, b, c_in);
    output sum, c_out;
    input a, b, c_in;
    wire s1, c1, c2;
    xor (s1, a, b);
    xor (sum, s1, c_in);
    and (c1, a, b);
    and (c2, s1, c_in);
    or (c_out, c1, c2);
endmodule
```



YouTube - PrepFusion - GATE

-> Click Here To Visit Our website : www.prepfusion.in

-> Click Here For Video Lectures ; YT @AnishSaha_

Gate-Level Modeling

2. 4-Bit Full Adder Design:

- Connects four 1-bit full adders in series.
- The carry-out from one adder becomes the carry-in for the next.

Verilog Implementation

```
Verilog
module fulladd4 (sum, c_out, a, b, c_in);
    output [3:0] sum;
    output c_out;
    input [3:0] a, b;
    input c_in;
    wire c1, c2, c3;
    fulladd fa0(sum[0], c1, a[0], b[0], c_in);
    fulladd fa1(sum[1], c2, a[1], b[1], c1);
    fulladd fa2(sum[2], c3, a[2], b[2], c2);
    fulladd fa3(sum[3], c_out, a[3], b[3], c3);
endmodule
```

```
// Set up variables
reg [3:0] A, B; // 4-bit input operands
reg CIN; // Carry input
wire [3:0] SUM; // 4-bit sum output
wire COUF; // Carry output
// Instantiate the 4-bit full adder module
fulladder4B1 FA0SUM, COUF, A, B, CIN);
// Set up monitoring for the signal values
initial begin
    $monitor($time, " A = %b, B = %b, CIN = %b -> COUF = %b, SUM = %b", A, B, CIN, COUF, SUM);
end
// Stimulate inputs
initial begin
    A = 4'd0; B = 4'd0; CIN = 1'b0;
    // Stimulate various input combinations with delays
    #5 A = 4'd0; B = 4'd4; CIN = 1'b0;
    #5 A = 4'd2; B = 4'd6; CIN = 1'b0;
    #5 A = 4'd4; B = 4'd6; CIN = 1'b0;
    #5 A = 4'd6; B = 4'd6; CIN = 1'b0;
    #5 A = 4'd0; B = 4'd0; CIN = 1'b0;
    // End simulation
    #5 $finish;
end
```

```
Time=0: A=0000, B=0000, C_IN=0, C_OUT=0, SUM=0000
Time=5: A=0011, B=0100, C_IN=0, C_OUT=0, SUM=0111
Time=10: A=0010, B=0101, C_IN=0, C_OUT=0, SUM=0111
Time=15: A=0011, B=0101, C_IN=0, C_OUT=1, SUM=0101
```

YouTube - PrepFusion - GATE

Gate-Level Modeling

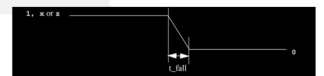
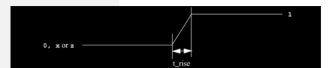
Gate Delays -> used for simulation

Gate delays in Verilog allow designers to model the timing behavior of circuits, specifying the time taken for signal transitions.

Rise, Fall, and Turn-off Delays

1. Types of Delays:

- Rise Delay:** Transition to logic 1 from any other value (0, x, or z).
- Fall Delay:** Transition to logic 0 from any other value (1, x, or z).
- Turn-off Delay:** Transition to high impedance (z) from any other value.



YouTube - PrepFusion - GATE

Verilog Framework

2. Delay Specifications:

- Single Delay:** Applied to all transitions (rise, fall, and turn-off).
- Two Delays:** Specifies separate rise and fall delays. Turn-off delay is the minimum of these two.
- Three Delays:** Specifies rise, fall, and turn-off delays independently.

Examples:

- Single delay: and #5 a1(out, i1, i2); (5 for all transitions).
- Two delays: and #4,6 a2(out, i1, i2); (rise = 4, fall = 6), *turn off time = 4*
- Three delays: buf10 a3(out, in, control); (rise = 3, fall = 4, turn-off = 5).

Verilog Framework

Min/Typ/Max Values

- Purpose:** Used to model the variability in gate delays caused by IC fabrication process differences. Designers can specify:

- Min Value:** Minimum expected delay.
- Typ Value:** Typical (default) delay value.
- Max Value:** Maximum expected delay.

2. Runtime Control:

- Select min, typ, or max at runtime using simulator options (=mindelays, +typdelays, or =maxdelays).
- Default is the **typ** delay if no option is specified.

```
// One delay
// if =mindelays, delay= 4
// if =typdelays, delay= 5
// if =maxdelays, delay= 6
and #(4:5:6) a1(out, i1, i2);

// Two delays
// if =mindelays, rise= 3, fall= 5, turn-off = min(3,5)
// if =typdelays, rise= 4, fall= 6, turn-off = min(4,6)
// if =maxdelays, rise= 5, fall= 7, turn-off = min(5,7)
and #(3:4:5, 5:6:7) a2(out, i1, i2);

// Three delays
// if =mindelays, rise= 2 fall= 3 turn-off = 4
// if =typdelays, rise= 3 fall= 4 turn-off = 5
// if =maxdelays, rise= 4 fall= 5 turn-off = 6
and #(2:3:4, 3:4:5, 4:5:6) a3(out, i1, i2);
```

Examples of invoking the Verilog-XL simulator with the command-line options are shown below. Assume that the module with delays is declared in the file test.v.

```
// Invoke simulation with maximum delay
> verilog test.v =mindelays
// Invoke simulation with minimum delay
> verilog test.v =mindelays
// Invoke simulation with typical delay
> verilog test.v =typdelays
```

YouTube - PrepFusion - GATE

YouTube - PrepFusion - GATE

-> Click Here To Visit Our website : www.prepfusion.in

-> Click Here For Video Lectures ; YT @AnishSaha_

Verilog Framework

Delay Example

1. Module Description:

- A simple combinational circuit is defined with the logic equation:
 $out = (a \& b) + c$.
- The circuit consists of two gates:
 - AND gate with delay = 5 units.
 - OR gate with delay = 4 units.

Verilog Definition:

```
Verilog
module D (out, a, b, c):
  output out;
  input a, b, c;
  wire e;
  and #5 a1(e, a, b); // Delay of 5
  or #4 o1(out, e, c); // Delay of 4
endmodule
```

Verilog

```
module stimulus;
  reg A, B, C;
  wire OUT;
  D d1 (OUT, A, B, C);

  initial begin
    A = 1'b0; B = 1'b0; C = 1'b0;
    #10 A = 1'b1; B = 1'b1; C = 1'b1;
    #10 A = 1'b1; B = 1'b0; C = 1'b0;
    #20 $finish;
  end
endmodule
```

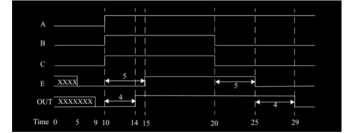
YouTube - PrepFusion - GATE

Verilog Framework

3. Simulation Output:

Waveforms:

- At time 10, A, B, and C transition to 1:
 - E changes to 1 after a delay of 5 units.
 - OUT changes to 1 after an additional delay of 4 units.
- At time 20, B and C transition to 0:
 - E changes to 0 after 5 units.
 - OUT transitions to 0 after an additional 4 units.



YouTube - PrepFusion - GATE

Exercise Chapter 5 : Verilog HDL by Samir Palnitkar

1: Create your own 2-input Verilog gates called my-or, my-and and my-not from 2-input nand gates. Check the functionality of these gates with a stimulus module.

2: A 2-input xor gate can be built from my-and, my-or and my-not gates. Construct an xor module in Verilog that realizes the logic function, $z = xy' + x'y$. Inputs are x and y, and z is the output. Write a stimulus module that exercises all four combinations of x and y inputs.

3: The 1-bit full adder described in the chapter can be expressed as a sum of products form.

$$sum = a \oplus b \oplus c$$
$$c_out = a \cdot b + b \cdot c + a \cdot c$$

Assuming a, b, c are the inputs and sum and c_out are the outputs, design a logic circuit to implement the 1-bit full adder, using only nand, nor, and or gates. Write the Verilog description for the circuit. You may use up to 4-input Verilog primitives and or gates. Write the stimulus for the full adder and check the functionality for all input combinations.

4: The logic diagram for an RS latch with delay is shown below.

YouTube - PrepFusion - GATE

instantiating the xor gates. Write the stimulus module for the RS latch, using the following table, and verify the outputs.

set	reset	Q
0	0	0
0	1	0
1	0	1
1	1	?

5: Design a 2-to-1 multiplexer using halfadd and halfsub gates as shown below.

The delay specifications for gates h1 and h2 are as follows:

The End



YouTube - PrepFusion - GATE

-> Click Here To Visit Our website : www.prepfusion.in

-> Click Here For Video Lectures ; YT @AnishSaha_

Lecture 7

Exercise 5 Solutions

YouTube - PrepFusion - GATE

```
module my_and(output Y, input A, B);
  wire nand_out;
  nand(nand_out, A, B);
  and(Y, nand_out, nand_out);
endmodule

module my_or(output Y, input A, B);
  wire nand_a, nand_b;
  nand(nand_a, A, A);
  nand(nand_b, B, B);
  or(Y, nand_a, nand_b);
endmodule

module my_not(output Y, input A);
  nand(Y, A, A);
endmodule
```

```
module chipsolutions_5;
  reg A, B;
  wire AND_OUT, OR_OUT, NOT_OUT;

  // Instantiate gates
  my_and a1(AND_OUT, A, B);
  my_or o1(OR_OUT, A, B);
  my_not n1(NOT_OUT, A);

  initial $monitor("%d-%d-%d | AND_OUT=%d OR_OUT=%d NOT_OUT=%d", A, B, AND_OUT, OR_OUT, NOT_OUT);

  initial begin
    $dumpfile("chipsolutions_5.vcd");
    $dumpvars(0, chipsolutions_5);

    A = 0; B = 0; #5;
    A = 0; B = 1; #5;
    A = 1; B = 0; #5;
    A = 1; B = 1; #5;

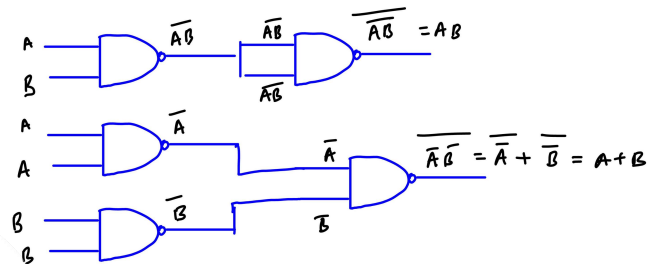
    $finish;
  end
endmodule
```

A=0 B=0	AND_OUT=0	OR_OUT=0	NOT_OUT=1
A=0 B=1	AND_OUT=0	OR_OUT=1	NOT_OUT=1
A=1 B=0	AND_OUT=0	OR_OUT=1	NOT_OUT=0
A=1 B=1	AND_OUT=1	OR_OUT=1	NOT_OUT=0

YouTube - PrepFusion - GATE

Question

1: Create your own 2-input Verilog gates called my-or, my-and and my-not from 2-input nand gates. Check the functionality of these gates with a stimulus module.



YouTube - PrepFusion - GATE

Question

2: A 2-input xor gate can be built from my-and, my-or and my-not gates. Construct an xor module in Verilog that realizes the logic function, $z = xy' + x'y$. Inputs are x and y, and z is the output. Write a stimulus module that exercises all four combinations of x and y inputs.

$$A \oplus B = A\bar{B} + \bar{A}B$$

YouTube - PrepFusion - GATE

-> Click Here To Visit Our website : www.prepfusion.in